

Σ ۲۱۰۱۳۳۹۵۱۶۱۰۱۰۱۰

۱۰۰۰۰۰۰۰۰۰۰۰۰۰

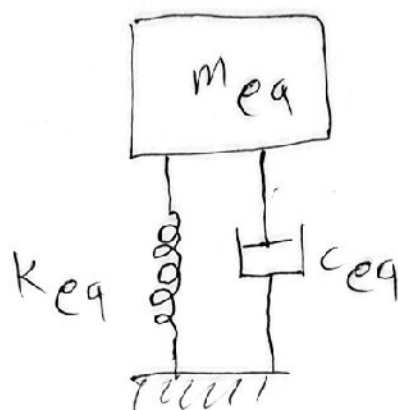
1 Mathematical Model of a motorcycle

1.1 mass: $m_{eq} = m_{wheels} + m_{vehicle} + m_{rider}$

stiffness: $K_{eq} = K_{tires} + K_{struts} + K_{rider}$

dampin: $c_{eq} = c_{struts} + c_{rider}$

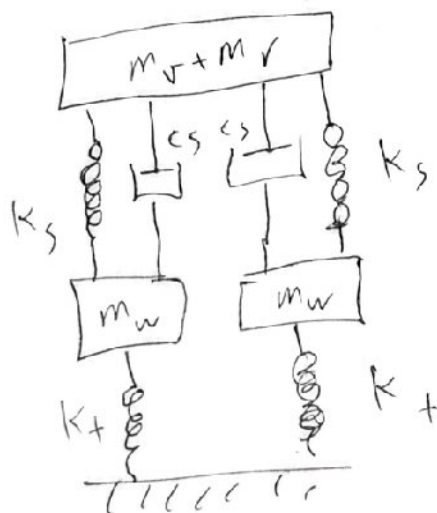
model a:



→ 1 degree of freedom

model b:

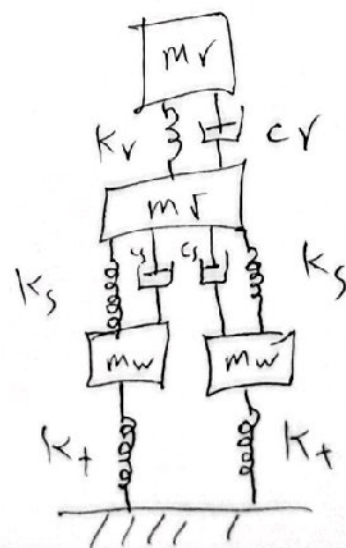
(springs masses are separated)



→ 2 degrees of freedom

model c:

all masses separated



3 degrees of freedom

1.2 if $x(t)$: vertical displacement of system

$y_r(t)$: vertical road displacement

→ model a:

$$F_s = -K_{eq}(x - y_r), \quad F_d = -c_{eq}(x' - y_r')$$

Newton
law $\rightarrow F_s + F_d = m_{eq} x'$

$$\rightarrow m_{eq} x'' + c_{eq}(x' - y_r') + K_{eq}(x - y_r) = 0$$

model b:

$$F_{tire} = -K_t(x_w - y_r), \quad F_{s,d} = -c_s(x_w' - x_s')$$

$$F_{s,s} = -K_s(x_w - x_s)$$

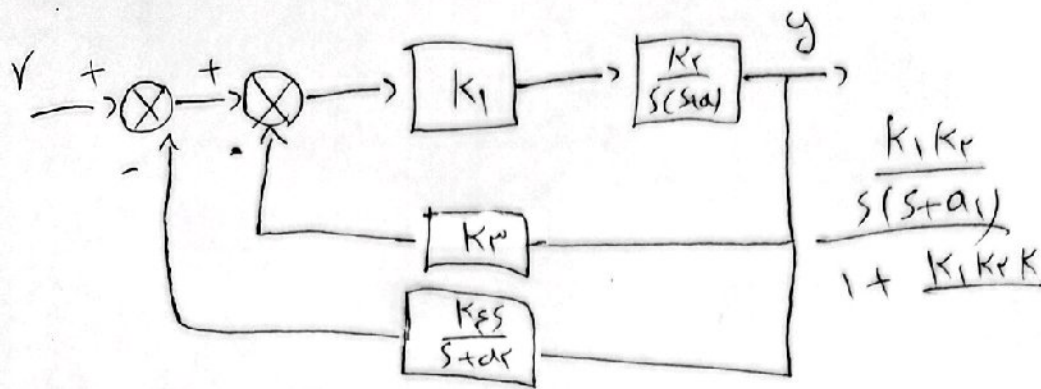
$$\rightarrow m_w x'' = -K_t(x_w - y_r) - K_s(x_w - x_s) - c_s(x_w' - x_s')$$

model c:

$$m_w x'' + (K_t + K_s)x_w + c_s x_w' - K_s x_s - c_s x_s' = K_t y_r$$

$$m_s x_s'' + K_s x_s + c_s x_s' - K_s x_w - c_s x_w' = 0$$

2 Human Eye Movement



inner loop:

$$\frac{\frac{K_1 K_r}{s(s+a_1)}}{1 + \frac{K_1 K_r K_p}{s(s+a_1)}} = \frac{K_1 K_r}{K_1 K_r K_p + s(s+a_1)}$$

outer loop Gain:

$$\frac{\frac{K_1 K_r}{K_1 K_r K_p + s(s+a_1)}}{1 + \frac{K_1 K_r K_p s}{(K_1 K_r K_p + s(s+a_1))(s+a_2)}} = \frac{K_1 K_r (s+a_2)}{(K_1 K_r K_p + s(s+a_1))(s+a_2) + K_1 K_r K_p s}$$

$$\rightarrow \frac{Y(s)}{R(s)} = \frac{K_1 K_r (s+a_2)}{s^3 + (a_1 + a_2)s^2 + (a_1 a_2 + K_1 K_r K_p)s + K_1 K_r K_p a_2}$$

2.2

$$A = a_1 + a_r, \quad B = a_1 a_r + k_1 k_p k_w, \quad c = k_1 k_p k_w a_r + k_1 k_p k_f$$

$$D = k_1 k_r, \quad E = k_1 k_r a_r$$

$$\rightarrow s^3 Y(s) + A s^2 Y(s) + B s Y(s) + C Y(s) = D s R(s) + E R(s)$$

$$\xrightarrow{\mathcal{L}^{-1}} \frac{d^3 y(t)}{dt^3} + A \frac{d^2 y}{dt^2} + B \frac{dy}{dt} + C y = D \frac{dV(t)}{dt} + E V(t)$$

$$R(s) \xrightarrow{\quad} \boxed{\frac{1}{s^3 + A s^2 + B s + C}} \xrightarrow{G(s)} \boxed{D s + E} \xrightarrow{\quad} Y(s)$$

$$y''' + A y'' + B y' + C y = V$$

$$x_1 = y, \quad x_2 = y', \quad x_3 = y''$$

$$\rightarrow x_1' = x_2, \quad x_2' = x_3, \quad x_3' = -C x_1 - B x_2 - A x_3 + V$$

$$\Rightarrow \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -C & -B & -A \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} V \quad \checkmark$$

$$\text{also: } Y(s) = (D s + E) G(s)$$

$$\Rightarrow y = D x_1' + E x_1 = D x_2 + E x_1$$

$$\rightarrow y = [E \ D] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \checkmark$$

3 Hybrid solar cell and Diesel power

$$3.1 \quad P_i = \frac{1}{C_f s} \times K_{Pr} \times \frac{K_{P_i} s + K_{id}}{s} \times \frac{1}{L + s} \times \frac{1}{C_f s}$$

$$\rightarrow P_i = \frac{K_{Pr} (K_{P_i} s + K_{id})}{L_f C_f s^5}, \quad P_r = \frac{K_{P_i} s + K_{id}}{L_f C_f s^3}, \quad P_w = \frac{L + s + R_f}{C_f L_f s^3}$$

$$L_i = -\frac{R_f}{L + s}, \quad L_r = -\frac{K_{P_i} s + K_{id}}{L_f s^3}, \quad L_w = \frac{K_{Pr} (K_{P_i} s + K_{id})}{L_f C_f s^5}$$

$$\Delta = 1 - (L_i + L_r + L_w), \quad \Delta_i = \Delta_r = \Delta_w = 1$$

$$T(s) = \frac{V_{load}}{I_{cf}} = \frac{\sum_k P_k \Delta_k}{\Delta} = \frac{P_i + P_r + P_w}{\Delta}$$

$$= \frac{K_{Pr} (K_{P_i} s + K_{id}) + s (K_{P_i} s + K_{id}) + (L + s + R_f) s^3}{L_f C_f s^5 + C_f R_f s^3 + C_f (K_{P_i} s + K_{id}) s^3 + K_{Pr} (K_{P_i} s + K_{id}) s}$$

$$\frac{V_{load}}{I_{dist}} : \quad P_i = \frac{1}{C_f s}, \quad L_r, L_w, L_w \text{ as before}$$

$$\therefore T(s) = \frac{V_{load}}{I_{dist}} = \frac{L + s^3}{L_f C_f s^3 + C_f R_f s^3 + C_f (K_{P_i} s + K_{id}) s^3 + K_{Pr} (K_{P_i} s + K_{id}) s}$$

3.2

$$\frac{V_{load}}{I_{ct}} = \frac{\overbrace{(L f)}^{\beta_r} s + \overbrace{(K p_1 + R f)}^{\beta_r} s + \overbrace{(K p_1 K_{pr} + K_{id})}^{\beta_1} s + \overbrace{(K_{pr} K_{id})}^{\beta_0}}{(1 + c f) s + \underbrace{(c + R f + c f K p_1)}_{A_r} s + \underbrace{(c f K_{id} + K_{pr} K_{pr})}_{A_r} s + \underbrace{(K_{pr} K_{id})}_{A_1} s}$$

$$= \frac{\frac{\overbrace{\beta_r}^{h_r}}{A_r} s + \frac{\overbrace{\beta_r}^{h_r}}{A_r} s + \frac{\overbrace{\beta_1}^{h_1}}{A_r} s + \frac{\overbrace{\beta_0}^{h_0}}{A_r} s}{s + \underbrace{\frac{A_r}{A_r}}_{a_r} s + \underbrace{\frac{A_r}{A_r}}_{a_r} s + \underbrace{\frac{A_1}{A_r}}_{a_1} s} = \frac{h_r s^3 + h_r s^2 + h_1 s + h_0}{s^4 + a_r s^3 + a_r s^2 + a_1 s}$$

$$\rightarrow \frac{d}{dt} \begin{pmatrix} q_1 \\ q_r \\ q_r \\ q_r \end{pmatrix} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -a_1 & -a_r & -a_r & -a_r \end{bmatrix}}_{A_1} \begin{pmatrix} q_1 \\ q_r \\ q_r \\ q_r \end{pmatrix} + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}}_{B_1} \bar{c} f$$

$$V_{load} = \underbrace{\begin{bmatrix} h_0 & h_1 & h_r & h_r \end{bmatrix}}_{C_1} \begin{pmatrix} q_1 \\ q_r \\ q_r \\ q_r \end{pmatrix}, \quad p_1 = 0$$

$$\frac{V_{load}}{I_{dist}} = \frac{(L f) s^r}{L + c f s^r + (c + R f + c f K p_1) s^r + (c f K_{id} + K_{pr} K_{pr}) s^r + (K_{pr} K_{id})}$$

$$= \frac{\overbrace{h_r}^{h_r'} \underbrace{\left(\frac{1}{c f} \right)}_{a_r'} s^r}{s^r + \underbrace{\left(\frac{c + R f + c f K p_1}{L + c f} \right)}_{a_r'} s^r + \underbrace{\left(\frac{c f K_{id} + K_{pr} K_{pr}}{L + c f} \right)}_{a_r'} s^r + \underbrace{\left(\frac{K_{pr} K_{id}}{L + c f} \right)}_{a_0'}}$$

$$\frac{d}{dt} \begin{pmatrix} q_w \\ q_g \\ q_v \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a'_2 - a'_1 & -a'_0 & 0 \end{pmatrix}}_{A_r} \begin{pmatrix} q_w \\ q_g \\ q_v \end{pmatrix} + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}}_{B_r} \dot{d}_{cst}$$

$$v_{load} = \underbrace{\begin{bmatrix} 0 & 0 & b'_1 \end{bmatrix}}_{C_r} \begin{pmatrix} q_w \\ q_g \\ q_v \end{pmatrix} \quad D_r = 0$$

$$\frac{d}{dt} x = A x + B \dot{c}_f(t) + w \dot{d}_{cst}(t)$$

$$v_{load}(t) = C x(t) + D \dot{c}_f + V \dot{d}_{cst}(t)$$

$$A = \begin{bmatrix} A_1 & 0 \\ 0 & A_r \end{bmatrix}, \quad B = \begin{bmatrix} B_1 \\ 0_{r \times 1} \end{bmatrix}, \quad w = \begin{bmatrix} 0_{1 \times 1} \\ B_r \end{bmatrix}$$

$$C = \begin{bmatrix} C_1 & C_r \end{bmatrix} \quad D = D_1$$

$$V = D_r$$

4 forced vibration with Coulomb damping

4.1 $m\ddot{x} = -kx + F \sin \omega_0 t \rightarrow m\ddot{x} + kx = F \sin \omega_0 t$

4.2

$$\dot{x}_1 = x_2$$

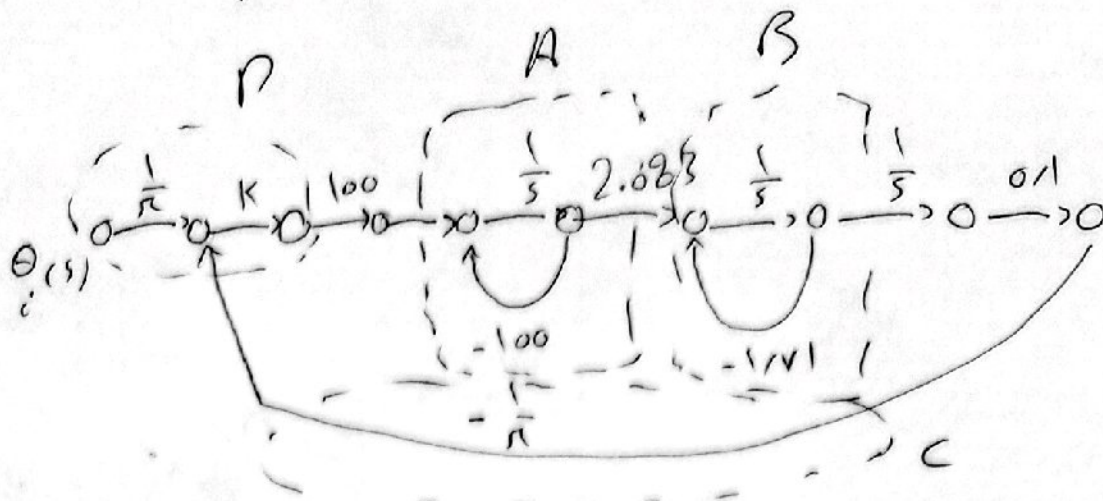
$$\ddot{x}_1 = \dot{x}_2 \rightarrow m\dot{x}_2 + kx_1 = F \sin \omega_0 t$$

$$= \frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} F \sin \omega_0 t$$

$$g = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

5 Antenna position control

5.1



A: controller, B = Power Amplifier and Motor

C = Feed back sensor, A = Command and Error detection

5.2

$$P_1 = \frac{V_0 / \Lambda^w K}{s^w \pi}, \quad L_1 = -\frac{100}{s}, \quad L_r = -\frac{1/v_1}{s}$$

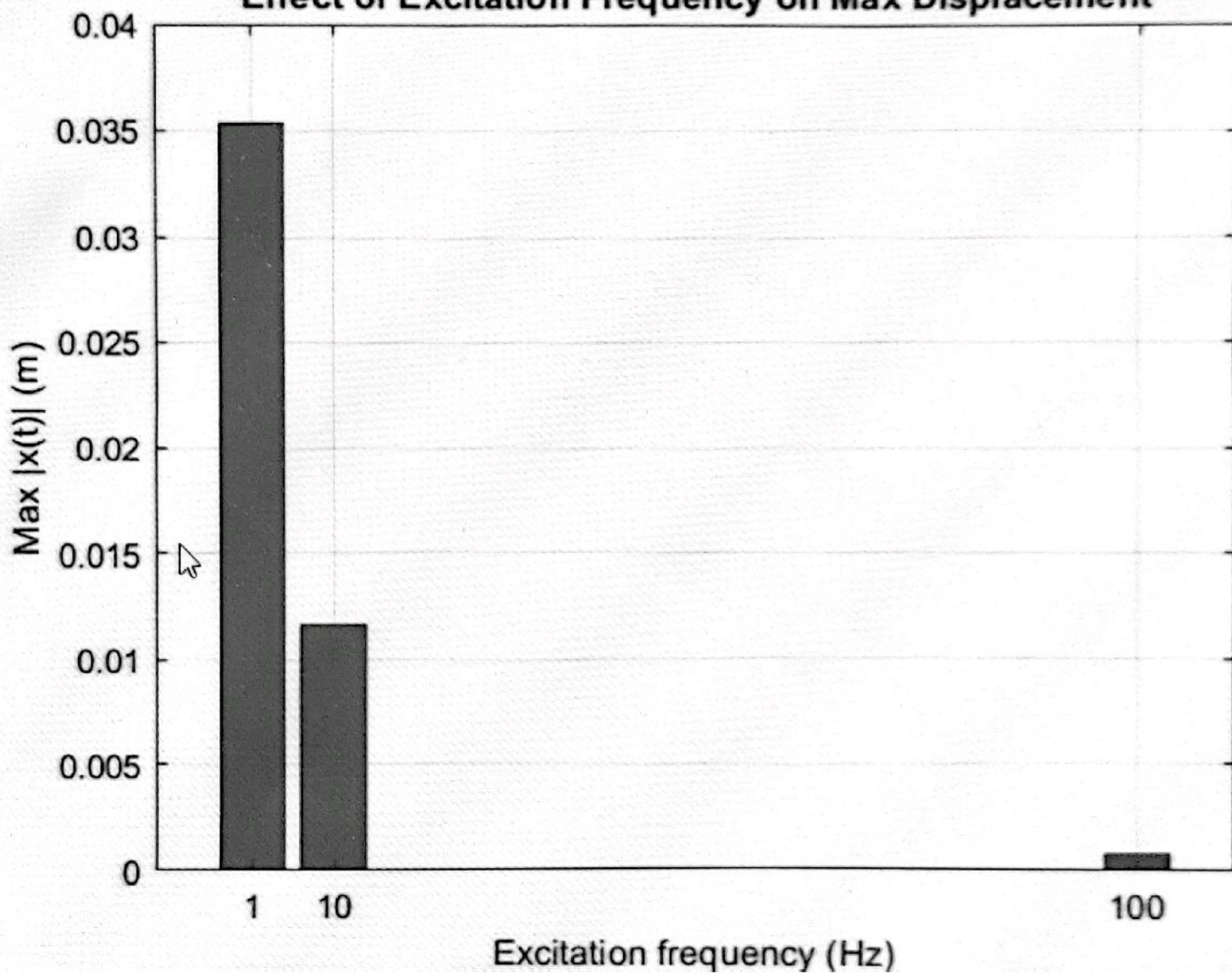
$$L_w = -\frac{V_0 / \Lambda^w K}{s^w \pi}, \quad L_1 L_r = \frac{1/v_1}{s^2}$$

$$\Delta(s) = 1 - (L_1 + L_r + L_w) + L_1 L_r = 1 + \frac{V_0 / \Lambda^w K}{s^w \pi} + \frac{100}{s} + \frac{1/v_1}{s} - \frac{1/v_1}{s}$$

$$\Delta = \frac{\pi s^w + V_0 / \Lambda^w K}{\pi s^w}, \quad \Delta_1 = 1$$

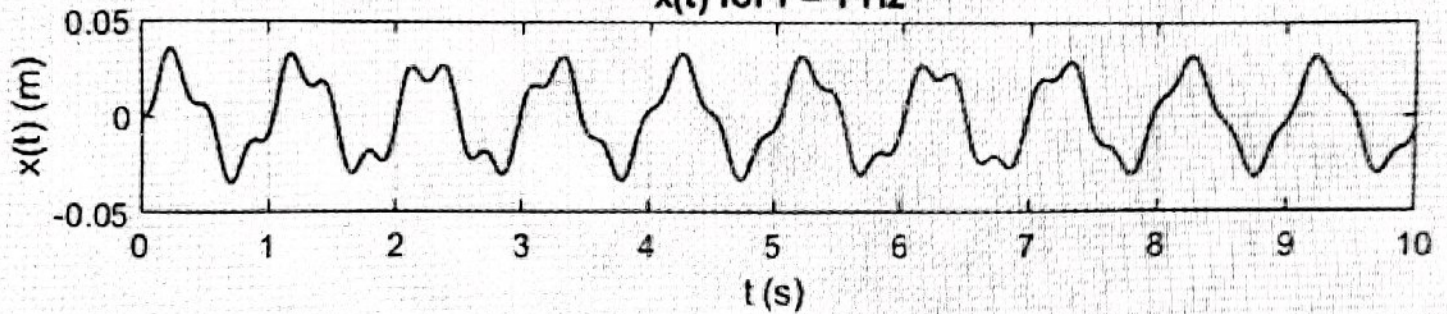
$$\rightarrow T(s) = \frac{P_1 \Delta_1}{\Delta} = \frac{V_0 / \Lambda^w K}{\pi s^w + 101/v_1 \pi s^2 + 1/v_1 \pi s + V_0 / \Lambda^w K}$$

Effect of Excitation Frequency on Max Displacement

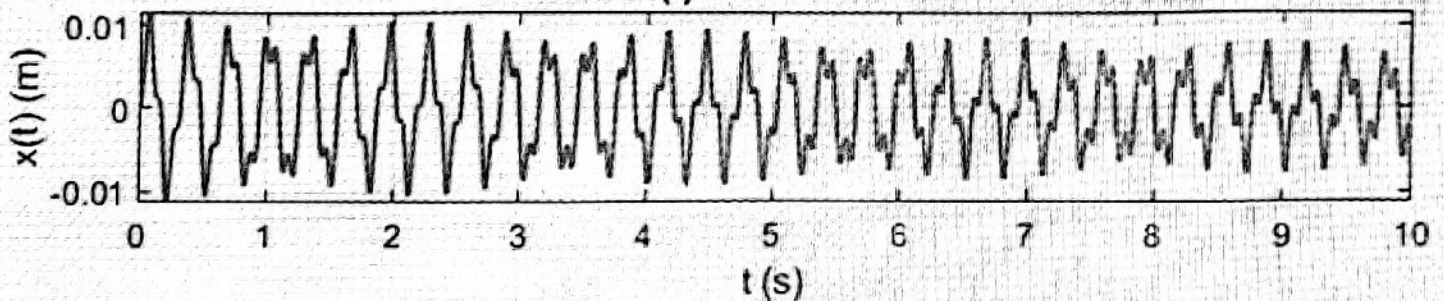


Nonlinear SMD Response for Different Excitation Frequencies

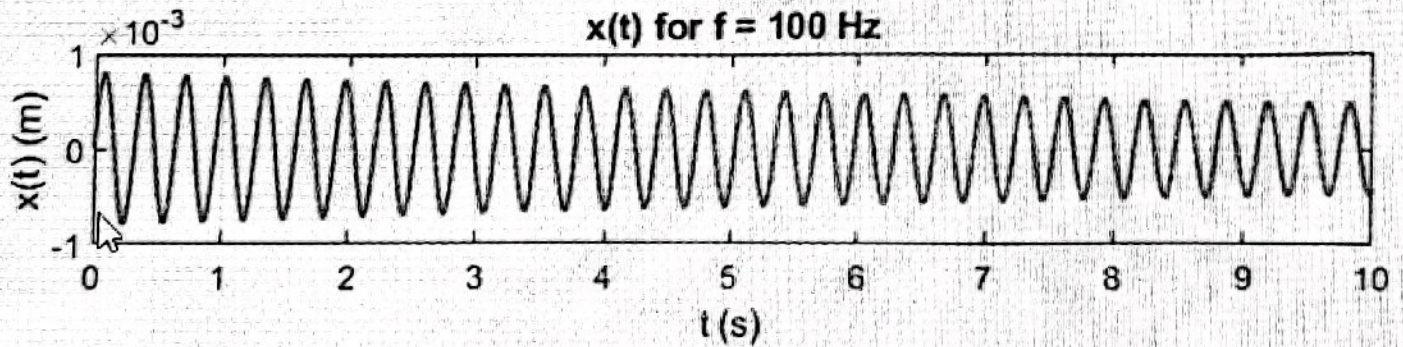
$x(t)$ for $f = 1$ Hz



$x(t)$ for $f = 10$ Hz



$x(t)$ for $f = 100$ Hz



```

clc; clear; close all;

% parameters
m = 10;
c = 1.2;
k1 = 4000;
k3 = 1e5;
F0 = 100;

freqs_Hz = [1, 10, 100];
omegas = 2*pi*freqs_Hz;

% Time span + initial conditions
tspan = [0, 10];
x0 = [0; 0];

% ODE
results = struct();

for i = 1:length(omegas)

    omega = omegas(i);
    freq = freqs_Hz(i);

    odefun = @(t, x) smd_nonlinear(t, x, omega, m, c, k1, k3, F0);

    % (V) Call ode45 and store results
    [t, X] = ode45(odefun, tspan, x0);

    x1 = X(:, 1);

    results(i).t = t;
    results(i).X = X;
    results(i).x1 = x1;
    results(i).freq_Hz = freq;
    results(i).omega = omega;
    results(i).max_amp = max(abs(x1));

    fprintf('f = %3.0f Hz, omega = %7.2f rad/s: max |x(t)| = %.5f m\n', ...
            freq, omega, results(i).max_amp);
end

f = 1 Hz, omega = 6.28 rad/s: max |x(t)| = 0.03540 m
f = 10 Hz, omega = 62.83 rad/s: max |x(t)| = 0.01160 m
f = 100 Hz, omega = 628.32 rad/s: max |x(t)| = 0.00082 m

```

```

% Plot displacement responses

```

```

figure;
for i = 1:length(results)
    subplot(length(results), 1, i);
    plot(results(i).t, results(i).x1, 'LineWidth', 1.2);
    grid on;
    title(sprintf('x(t) for f = %g Hz', results(i).freq_Hz));
    xlabel('t (s)');
    ylabel('x(t) (m)');
end
sgtitle('Nonlinear SMD Response for Different Excitation Frequencies');

% Compare maximum amplitudes
figure;
amps = [results.max_amp];
bar(freqs_Hz, amps);
grid on;
xlabel('Excitation frequency (Hz)');
ylabel('Max |x(t)| (m)');
title('Effect of Excitation Frequency on Max Displacement');

% Local ODE
function dxdt = smd_nonlinear(t, x, omega, m, c, k1, k3, F0)
    x1 = x(1);
    x2 = x(2);

    dx1dt = x2;
    dx2dt = -(c/m)*x2 - (k1/m)*x1 - (k3/m)*x1^3 + (F0/m)*sin(omega * t);

    dxdt = [dx1dt;
            dx2dt];
end

```

In this part we simulated the response of a nonlinear spring–mass–damper system excited by a harmonic force. The original second-order differential equation was rewritten in state-space form by defining displacement as the first state and velocity as the second. This allowed the system to be implemented in the standard ode45 pipeline used throughout the assignment.

Following the prescribed procedure, we first constructed the nonlinear state equations, then selected the simulation interval from 0 to 10 seconds and applied zero initial conditions. The harmonic excitation was included directly in the state equations, and a MATLAB function was created to evaluate the dynamics. We then used ode45 to integrate the system numerically for three different excitation frequencies: 1 Hz, 10 Hz, and 100 Hz.

The resulting displacement responses show clear differences across the three cases, both in amplitude and waveform shape. Because the system contains a cubic nonlinear stiffness term, the motion departs from a simple sinusoidal pattern, especially at higher frequencies. Overall, the results demonstrate how excitation frequency influences the behavior of a nonlinear vibratory system and illustrate the effectiveness of the ode45 workflow for analyzing such systems.

```

clc; clear; close all;

k = 1;
zeta = 0.7;
wn = 3;

% Denominator
den_T = [1 2*zeta*wn wn^2];
num_T = [K];

% Create transfer function object
sys_tf = tf(num_T, den_T);

disp('Transfer function model sys\_tf:');

```

Transfer function model sys_tf:

```
sys_tf
```

```
sys_tf =
```

$$\frac{1}{s^2 + 4.2s + 9}$$

Continuous-time transfer function.
Model Properties

```
%% 2) Convert the transfer function model to a state-space model
```

```

sys_ss = ss(sys_tf);

disp('State-space model sys\_ss (A, B, C, D):');

```

State-space model sys_ss (A, B, C, D):

```
A = sys_ss.A
```

```

A = 2x2
   -4.2000   -2.2500
    4.0000         0

```

```
B = sys_ss.B
```

```

B = 2x1
    0.5000
         0

```

```
C = sys_ss.C
```

```

C = 1x2
         0    0.5000

```

```
D = sys_ss.D
```

D = 0

```
%% 3) Define time vector and different input signals (step and sine)

Tend = 10;
Ts    = 0.001;
t     = 0:Ts:Tend;

% (a) Step input
u_step = ones(size(t));

% (b) Sine input
f_sin = 1;
u_sin = sin(2*pi*f_sin*t);

%% 4) Use lsim with transfer function model
-
% Step response
y_step_tf = lsim(sys_tf, u_step, t);

% Sine response
y_sin_tf = lsim(sys_tf, u_sin, t);

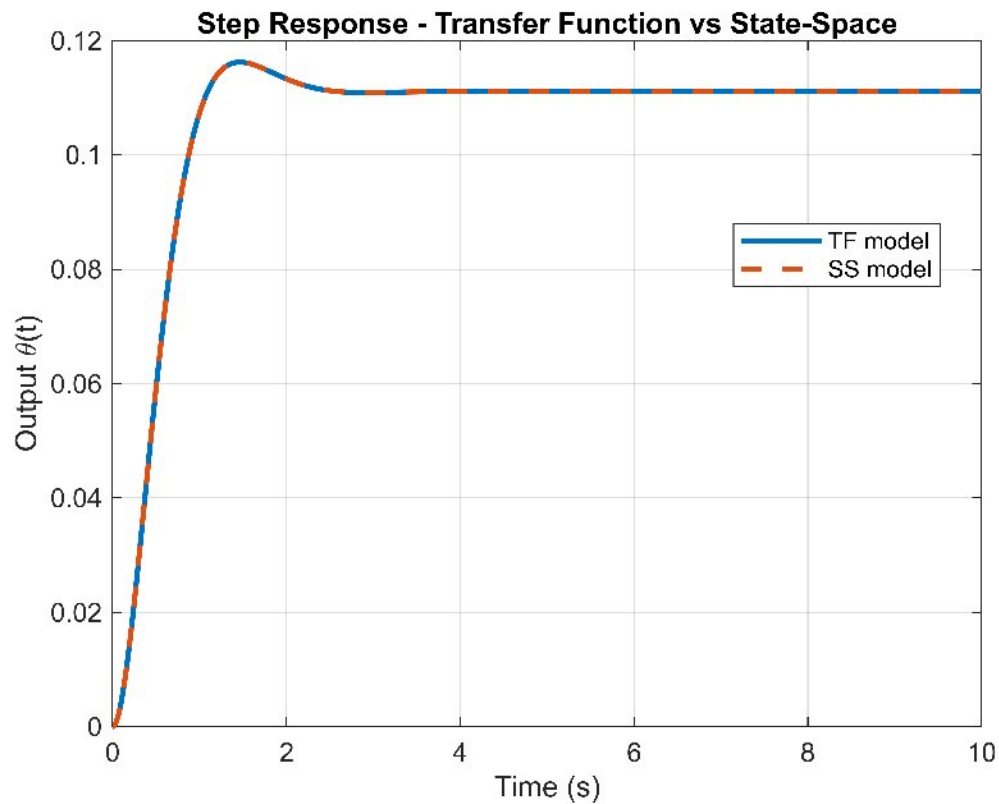
%% 5) Use lsim with state-space model (same inputs)

% Optional: we can also specify initial conditions x0 (here we use zero)
x0 = [0; 0]; % for a 2-state system

y_step_ss = lsim(sys_ss, u_step, t, x0);
y_sin_ss  = lsim(sys_ss, u_sin, t, x0);

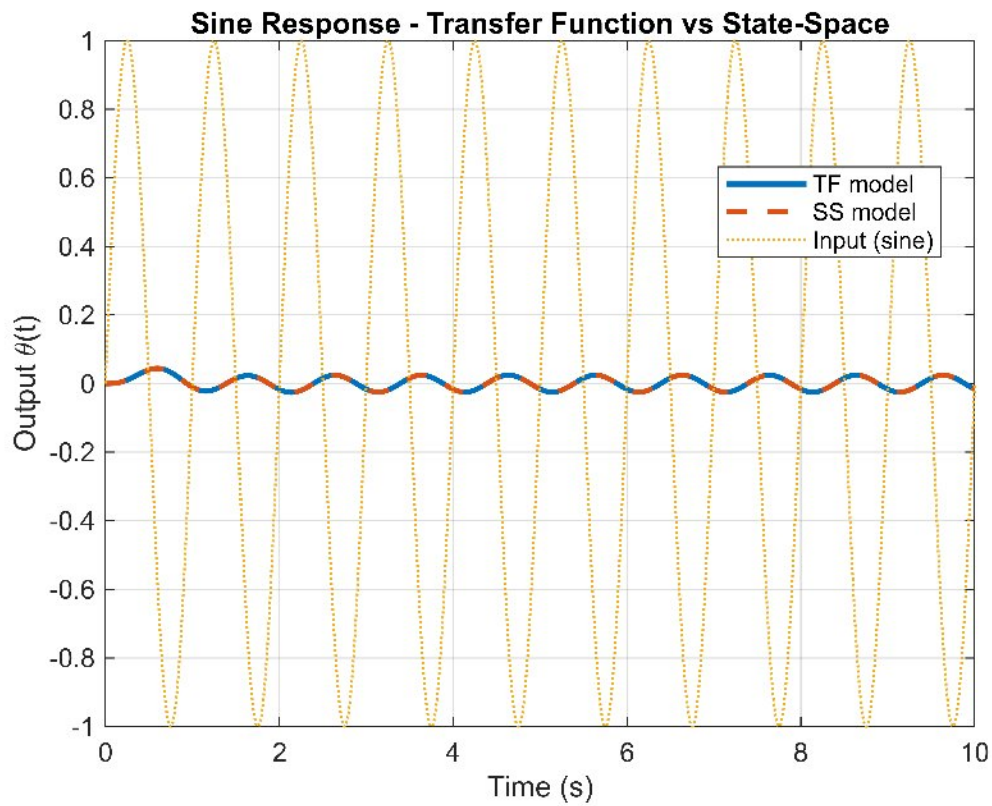
%% 6) Plot results and compare TF vs SS for step input

figure;
plot(t, y_step_tf, 'LineWidth', 2); hold on;
plot(t, y_step_ss, '--', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Output \theta(t)');
title('Step Response - Transfer Function vs State-Space');
legend('TF model', 'SS model', 'Location', 'Best');
grid on;
```



```
%% 7) Plot results and compare TF vs SS for sine input
```

```
figure;
plot(t, y_sin_tf, 'LineWidth', 2); hold on;
plot(t, y_sin_ss, '--', 'LineWidth', 2);
plot(t, u_sin, ':', 'LineWidth', 1); % show input too (optional)
xlabel('Time (s)');
ylabel('Output \theta(t)');
title('Sine Response - Transfer Function vs State-Space');
legend('TF model', 'SS model', 'Input (sine)', 'Location', 'Best');
grid on;
```



```
%% 8) Extra: separate plots
```

```
% Step response - TF only
```

```
figure;
```

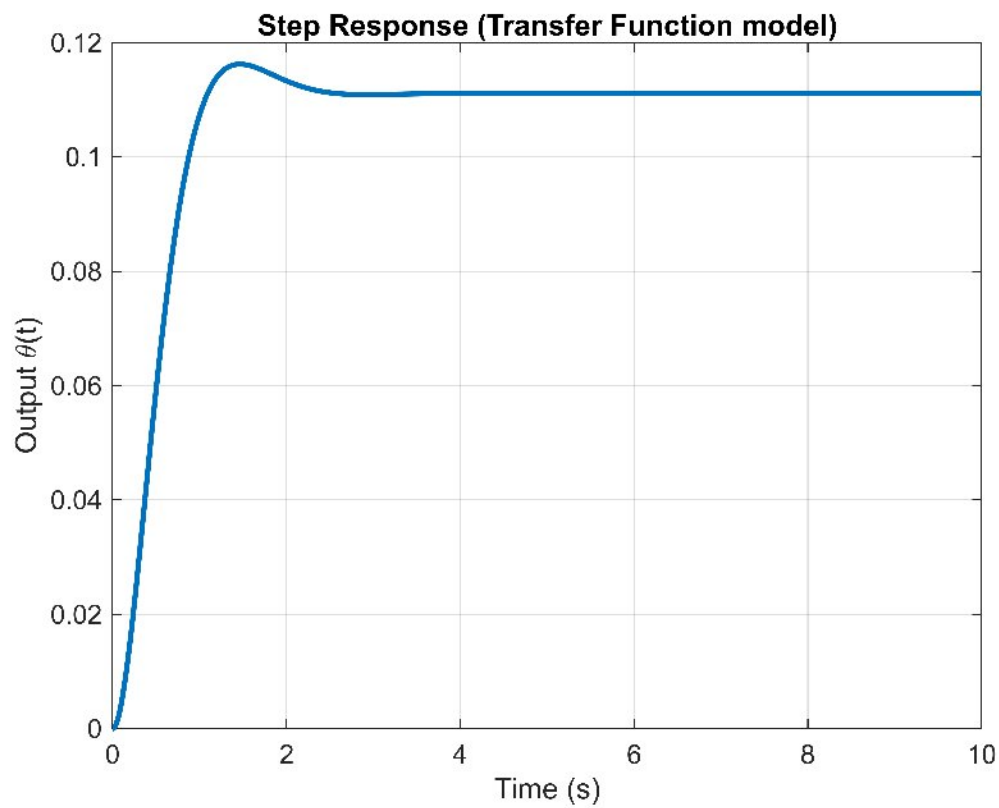
```
plot(t, y_step_tf, 'LineWidth', 2);
```

```
xlabel('Time (s)');
```

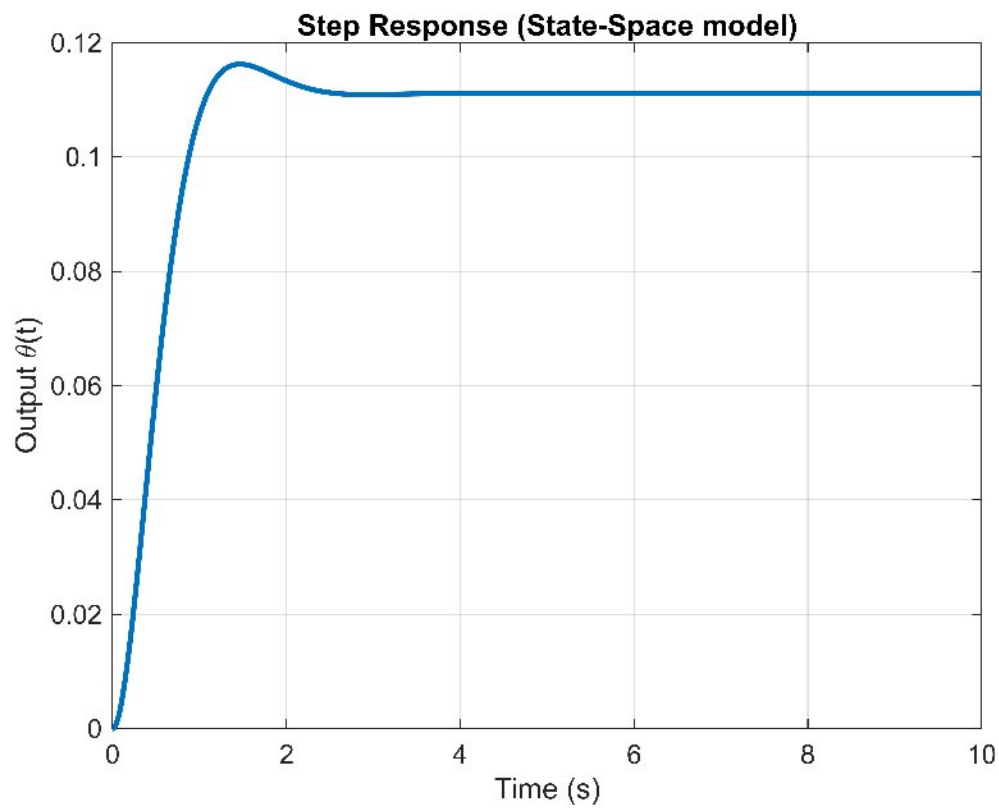
```
ylabel('Output \theta(t)');
```

```
title('Step Response (Transfer Function model)');
```

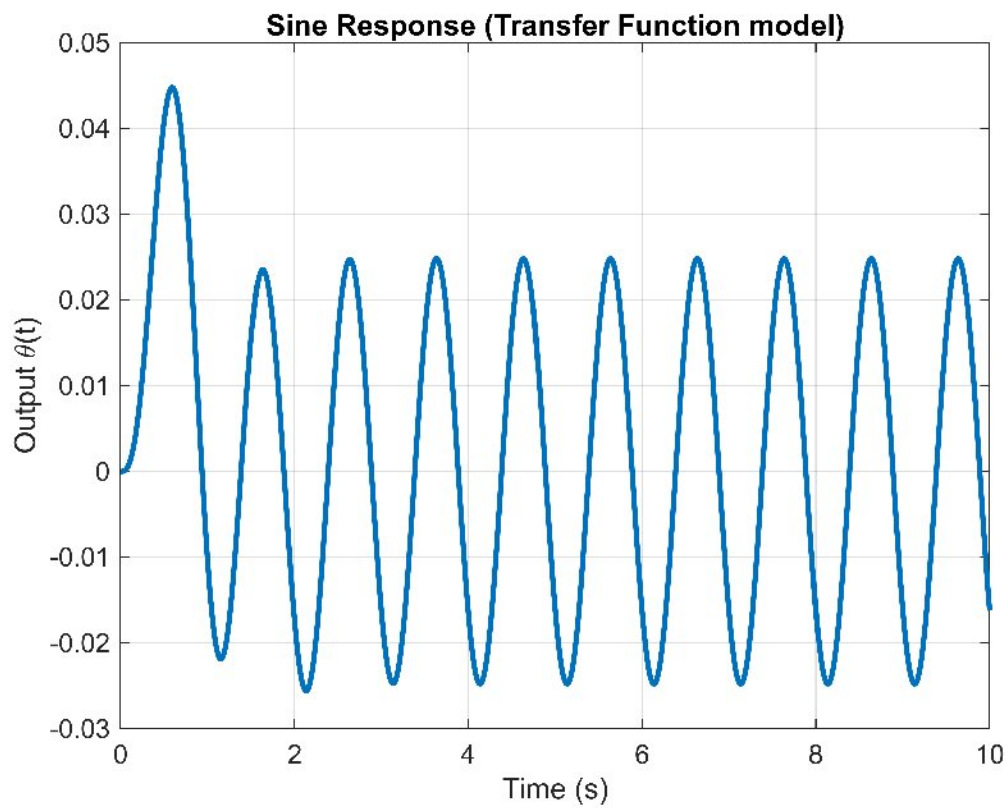
```
grid on;
```



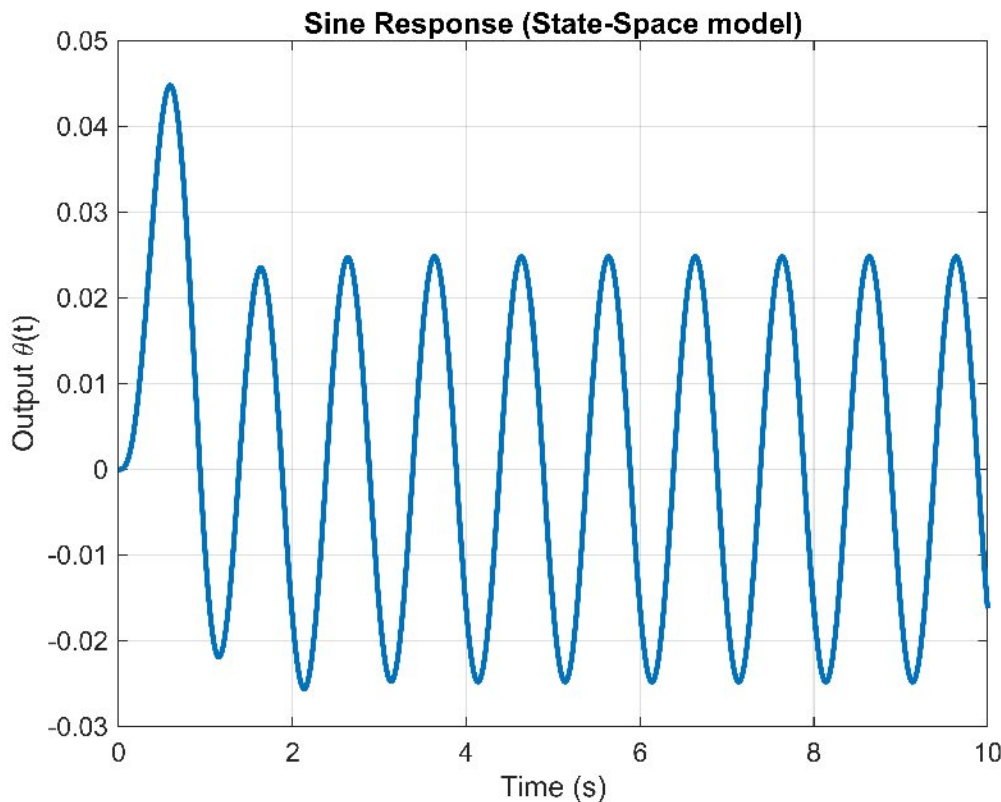
```
% Step response - SS only
figure;
plot(t, y_step_ss, 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Output \theta(t)');
title('Step Response (State-Space model)');
grid on;
```



```
% Sine response - TF only
figure;
plot(t, y_sin_tf, 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Output \theta(t)');
title('Sine Response (Transfer Function model)');
grid on;
```



```
% Sine response - SS only
figure;
plot(t, y_sin_ss, 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Output \theta(t)');
title('Sine Response (State-Space model)');
grid on;
```



The system in the figure represents a classic servo control loop, consisting of several subsystems: an amplifier, a motor, a gear train, an inertia, and a feedback sensor. When combined, these subsystems form a complete closed-loop LTI system whose response to different inputs can be analyzed.

MATLAB provides two equivalent ways to represent such a system:

1. Transfer Function (TF) model using `'tf()'`
2. State-Space (SS) model using `'ss()'`

we construct both models and show that they generate the same time response when simulated with `'lsim'`.

Why We Use LTI Objects

An LTI object in MATLAB is a mathematical representation of a system that allows:

automatic simplification

interconnection (series, parallel, feedback)

direct simulation

easy conversion between TF, SS, and ZPK forms

For example:

``tf()'` creates a transfer function object

``ss()'` creates a state-space object

``feedback()'` forms a closed-loop system automatically

This makes system modeling extremely flexible and avoids manual algebra.

The exact parameters from Figure 5 are not numerically given, so we use a reasonable second-order model that behaves like a typical antenna azimuth system. (If the exact transfer function from Q5.2 is available from class, it can replace this model without changing the workflow.)

The important part is learning the method, not the specific numeric values

Time Vector and Input Signals

The assignment requires simulation over 10 seconds for different inputs:

1. Step input

- * used to check tracking performance

- * represents a sudden command to “move the antenna to a new position”

2. Sinusoidal input

- * tests how the antenna responds to oscillatory disturbances or periodic commands

- * uses a 1 Hz sine wave

In MATLAB, both inputs are created as arrays matching the time vector so they can be passed into ``lsim'`.

Using ``lsim'`

The function `lsim` numerically computes the time response of an LTI system to arbitrary inputs:

```
[  
y(t) = \texttt{lsim(system,; input,; time,; initial_conditions)}  
]
```

It works with both:

Transfer function models, and

State-space models

This allows us to verify that the two system representations produce identical results, as theory predicts.

What the Generated Figures Show

several figures in this assignment:

1. Step Response – Transfer Function Model

This plot shows how the antenna reacts when it suddenly receives a new position command.

You will see behaviors such as:

rise time

settling time

overshoot

steady-state value

This response helps evaluate the control system's accuracy and speed.

2. Step Response – State-Space Model

This plot shows the same system, but simulated using the state-space representation.

The curve should perfectly overlap the TF step response.

This confirms:

Transfer function and state-space models are mathematically equivalent.

3. Sine Response – Transfer Function Model

This figure shows how the system responds to a sinusoidal input.

It reveals frequency-dependent behavior:

amplitude attenuation

phase delay

possible resonance

Such responses are important in antenna control because wind, vibration, and oscillatory commands can excite periodic motion.

Sine Response – State-Space Model

Again, identical to the TF response. The overlap demonstrates correct modeling and validates the SS form.

Comparison Plots

Some figures intentionally plot **both TF and SS responses on the same axes**, showing the curves lying exactly on top of each other. These comparison plots are essential for your assignment because they directly prove: The LTI system behaves the same regardless of whether it is represented by transfer functions or state-space equations.

Conclusion

This assignment demonstrates the full workflow for simulating LTI systems in MATLAB:

creating LTI models (`'tf'`, `'ss'`)

generating input signals (step and sine)

simulating with `'lsim'`

comparing representations

The plots verify that the transfer function model and the state-space model are completely consistent, as expected from linear system theory. This confirms that MATLAB's LTI framework is reliable for modeling and analyzing control systems such as the antenna azimuth mechanism in Figure 5.